TECHNICAL REPORT

Coin Challenge Multimodal Questioner

System design and engineering improvements

Project: `coin_challenge`

Date: 2026-08-16

An evidence-grounded questioner for interactive image matching, combining multimodal perception with deterministic protocol validation, bounded context, and provider-independent model access.

Architecture · Memory · Evidence · Reliability

Abstract

This project implements a multimodal questioner for an interactive image-matching task. At each decision step, the questioner receives a textual description of a hidden target, one visible candidate image, and the most recent oracle answer. It must either ask a concise visual question about the hidden target or conclude whether the candidate matches the target.

The current implementation uses an image-capable Doubao/Volcano Engine ModelArk endpoint as the default multimodal reasoning backend. A deterministic controller surrounds the model with structured memory, evidence provenance, answer-polarity parsing, conclusion gates, output repair, and bounded retries. The design intentionally separates perception from control: the model interprets images and proposes discriminative attributes, while program logic enforces the protocol and evidence requirements.

This report describes the architecture, technologies, engineering improvements, limitations, and future directions. It intentionally contains no benchmark scores or experimental measurements.

1. Problem Definition

An episode contains one fixed hidden target and an ordered sequence of candidate images. The questioner can observe:

- the textual target description;
- the current candidate image;
- an oracle answer only after asking a question.

At every step, the questioner must return exactly one of two actions:

1. ask an atomic visual question about the hidden target; or
2. return conclusion `0` for non-match or `1` for match.

Every action contains `question`, `conclusion`, and `reasoning`. Exactly one of `question` and `conclusion` must be non-null. A correct conclusion advances to the next candidate, while an incorrect conclusion terminates the episode.

The main engineering challenges are:

- descriptions may contain only a broad category;
- the questioner and oracle have different image permissions;
- target knowledge must persist across candidate changes;
- model output may violate JSON or action constraints;
- asking questions has a cost, but unsupported conclusions are riskier;
- remote multimodal APIs may time out, disconnect, or reject requests.

2. System Architecture

The repository uses a layered architecture in which the environment owns the protocol, the model owns visual perception, and the controller owns evidence consistency.

```
eval_model.py
    |
```

```
    +-- selects dataset split and description type
    +-- creates questioner and oracle clients
    +-- runs episodes and writes result files
    |
    v
QAEnv (env.py)
    |
    +-- exposes the candidate RGB image and target description
    +-- validates actions and computes rewards
    +-- switches candidates and invokes the oracle
    |
    v
YourQuestioner (Questioner.py)
    |
    +-- invokes the candidate-side vision model
    +-- maintains target and candidate memory
    +-- validates evidence and repairs actions
    |
    v
Model clients (utils.py)
    +-- Doubao / ModelArk
    +-- Gemini
    +-- local vLLM
```

2.1 Module Responsibilities

- `Questioner.py` defines the questioner contract and the active `YourQuestioner` implementation.
- `env.py` implements the Gymnasium environment, reward logic, candidate transitions, image loading, and oracle calls.
- `utils.py` provides remote and local multimodal client abstractions.
- `eval_model.py` implements provider selection, CLI handling, the evaluation loop, summaries, and result serialization.
- `Oracle.py` defines the minimal oracle interface.
- `tests/` covers memory, evidence gates, action repair, candidate transitions, and retry classification.

3. Core Technologies

3.1 Python, Gymnasium, NumPy, and Pillow

The environment inherits from Gymnasium `Env` and models the episode lifecycle through `reset()` and `step()`. NumPy carries RGB image arrays between the environment and questioner. Pillow loads image assets and supports image encoding for remote requests and result files.

The environment is independent of a specific model provider. Questioner and oracle implementations can therefore change without modifying the reward or transition protocol.

3.2 Multimodal Model Access

Doubao is accessed through the OpenAI-compatible Volcano Engine ModelArk API. A request contains one textual prompt and one image. The repository also retains Gemini and local vLLM clients behind the same conceptual interface:

```
ask(*, prompt: str, images: list) -> str
```

Each multimodal request supports exactly one image. This restriction also reinforces the permission boundary: the questioner sees only the candidate image, and the oracle sees only the hidden target image.

3.3 Compact JSON Action Protocol

The model is instructed to return one compact JSON object. A question action has the following shape:

```
{"action":"question","attribute":"finish_color","question":"...","fallback":0,"reasoning":"..."}
```

A conclusion action has the following shape:

```
{"action":"conclusion","conclusion":0,"matches":["finish_color"],"conflicts":[],"reasoning":"..."}
```

The parser uses `json.JSONDecoder.raw_decode()` to locate the first action-bearing JSON object. This approach is safer than greedy regular expressions or delimiter slicing. The parser also accepts an older nested schema to preserve compatibility with earlier model output.

3.4 Candidate Image Fingerprints

The questioner computes a BLAKE2b digest from the image shape, data type, and pixel bytes. A changed digest means that the environment has advanced to a new candidate. Candidate-local state is then cleared while episode-level target evidence remains available.

3.5 Bounded Retry Policies

Questioner and oracle calls retry only transient connection failures and timeouts. Authentication, quota, parameter, and other non-transient errors fail immediately. This prevents a brief network interruption from destroying an episode while avoiding unbounded waits and repeated billing for permanent failures.

4. Questioner Design

4.1 Two-Level State Model

State is divided into target-level and candidate-level data.

Target-level state persists for the complete episode:

- the original target description;
- oracle question-and-answer evidence;
- normalized target facts;
- asked questions and attributes;
- answered attribute keys.

Candidate-level state is reset whenever the image fingerprint changes:

- the current candidate profile;
- the model's current comparison;
- the per-candidate question count;
- the candidate fingerprint.

This split prevents visual facts from one candidate from leaking into the next while preserving knowledge learned about the hidden target.

4.2 Structured Evidence Ledger

Every oracle answer is stored with the following fields:

```
question
answer
attribute
polarity: yes | no | uncertain
candidate_fingerprint
has_target_detail
```

The fingerprint identifies whether evidence directly concerns the current candidate. `has_target_detail` records whether the answer contains information beyond a bare yes or no that may support comparisons with later candidates.

Answer polarity is parsed conservatively. Clear yes/no prefixes map to `yes` or `no`. Phrases such as unknown, unclear, unsure, or not visible map to `uncertain`. Unrecognized answers also become `uncertain`, preventing ambiguous text from being treated as positive evidence.

4.3 Deterministic Evidence Rules

Questions are required to express one proposition that is visibly true of the current candidate and ask whether it is also true of the hidden target. Under this contract:

- one reliable current-candidate `no` is a direct visual conflict and supports conclusion `0`;
- for a category-only description, two independent high-discrimination `yes` answers with no conflict support conclusion `1`;
- `uncertain` supports neither outcome.

These rules are implemented in program logic and do not depend on the model using identical wording in its `matches` or `conflicts` arrays.

4.4 Grounded Positive Gate

For category-only descriptions, a positive conclusion must satisfy all of the following conditions:

- no direct current-candidate conflict exists;
- no unresolved model-reported conflict exists;
- at least two independent, non-category, non-generic attributes match;
- each matching attribute is grounded in oracle evidence or a target fact.

An attribute being answered is not equivalent to that attribute matching. The gate prevents the model from treating any previously discussed attribute as positive support.

4.5 Negative Gate

For a same-category candidate under a weak description, an early negative conclusion is rejected when the model provides no concrete conflict and question budget remains. The controller asks the model to produce a new discriminative question instead. A clear category mismatch or reliable target conflict allows an immediate negative conclusion.

4.6 Question Selection Policy

Questions must:

- concern only the hidden target;

- contain one atomic yes/no proposition;
- avoid image artifacts and unavailable information;
- avoid previously asked questions and attributes;
- prioritize color, component count, structure, border style, control layout, accessories, and surrounding objects;
- avoid generic categories and object types already provided by the description.

The default limit is two questions per candidate. The count resets after a candidate transition.

4.7 Output Repair and Fallback

The controller rejects:

- malformed or missing JSON actions;
- questions without an attribute key;
- repeated questions or repeated attributes;
- questions beyond the per-candidate limit;
- positive conclusions without grounded support;
- premature same-category negative conclusions without a conflict.

The rejection reason and a bounded summary of the previous response are added to one repair prompt. When question budget is exhausted, the repair prompt requires a conclusion. When an unsupported conclusion appears while budget remains, it requires a new question. If repair still fails, the controller applies a conservative fallback that remains subject to positive and negative evidence gates.

5. Context and Memory Management

Memory is serialized as compact JSON and includes the target description, target facts, oracle evidence, current candidate profile, current comparison, and previously asked attributes.

Default limits are:

- two questions per candidate;
- twelve retained evidence items;
- six thousand memory characters.

When memory exceeds the budget, the controller progressively removes old question text, older raw evidence, comparison details, and older facts. It retains a minimum amount of high-value evidence where possible. The bounded representation is suitable for remote models and local models with relatively small context windows.

Structured memory provides several advantages over a raw conversation transcript:

- target and candidate facts have explicit ownership;
- evidence provenance remains inspectable;
- lower-priority fields can be pruned independently;
- deterministic gates can consume the same evidence;
- the model does not need to repeatedly reinterpret a long dialogue.

6. Provider and Configuration Design

`utils.py` exposes a common remote-model abstraction and creates ModelArk clients through `create_doubao_client()`. Configuration loads from the repository `.env` file and falls back to the legacy `$HOME/.env.ml` path when necessary.

The configuration surface includes:

- API key and image-capable endpoint ID;
- API base URL;
- request timeout;
- output-length limit;
- provider-specific thinking control;
- independent questioner and oracle providers;
- an optional questioner-specific model ID.

The OpenAI client uses a bounded timeout and disables automatic SDK retries. The application layer then decides whether an exception is transient, avoiding compounded retries across layers.

7. Environment and Evaluation Engineering

7.1 Action Validation

The environment verifies that `question` and `conclusion` are present and uses an exclusive-or condition to guarantee that exactly one is non-null. The questioner contract requires integer conclusions even though the environment eventually compares their Boolean meaning with the candidate label.

7.2 Safe Image Loading

Before invoking Pillow, the environment reads the beginning of each asset and checks for a Git LFS pointer. A pointer produces an actionable recovery message instead of an opaque image-format exception.

7.3 Reproducible Dataset Splits

The evaluator supports `train`, `val`, and `test` datasets and six description types. A split script uses a fixed random seed to generate reproducible training, validation, and test files while preserving the original training dataset.

7.4 Result Serialization

Each description type produces a separate gzip-compressed JSON file. Results include actions, answers, reasoning, candidate observations, conclusion counts, question counts, reward values, full-episode status, and model timing. The CLI also prints an aggregate summary.

8. Main Engineering Improvements

8.1 From Template to Complete Agent

The initial questioner contained only interfaces and incomplete placeholders. `YourQuestioner` now provides a complete loop for prompt construction, model invocation, parsing, state updates, validation, repair, and fallback.

8.2 From Raw Dialogue to Structured Evidence

Raw question-and-answer text required the model to reinterpret all history and could not be verified by program logic. The evidence ledger adds attributes, polarity, candidate provenance, and target-detail markers so that important conclusions can be audited.

8.3 From Model-Only Decisions to Hybrid Control

The model handles visual interpretation and proposes discriminative attributes. The controller handles protocol rules, evidence thresholds, and deterministic outcomes. This hybrid approach reduces the direct impact of formatting variation and unsupported reasoning.

8.4 Explicit Provider Selection

Questioner and oracle backends can be selected independently from Doubao, Gemini, and local vLLM. The default path uses Doubao without requiring source edits, while compatibility options remain available.

8.5 Explicit Failure Boundaries

Transient network problems receive bounded retries. Authentication, quota, and parameter errors surface immediately. Invalid model actions receive one repair attempt. Invalid image assets receive a recovery instruction. Each failure class therefore has a predictable boundary.

8.6 Improved Testability

Model clients can be replaced by a dependency-injected fake client. Offline tests cover target memory across candidate transitions, question limits, repeated questions, attribute aliases, positive and negative evidence gates, output repair, memory budgets, and retry classification.

9. Security and Operations

- API keys are loaded only from environment variables or dotenv files.
- `.env` is excluded from version control.
- logs do not print secret values.
- results, logs, environments, and download caches are local artifacts.
- long evaluations should start with `nohup` and redirected output.
- paid evaluations should begin with a small slice.
- remote questioner and oracle calls must account for both API cost and endpoint rate limits.

10. Known Limitations

1. Results are written only after one complete description type, so there is no per-episode checkpoint.
2. Answer polarity parsing primarily targets English yes/no oracle responses.

3. The fixed two-question budget does not adapt to description richness.
4. Cross-candidate matching still relies partly on the model aligning target text with the current image.
5. Image fingerprinting reads all pixels and adds CPU cost for very large images.
6. The local client currently targets `localhost:8000/v1`.
7. Legacy launch scripts contain obsolete argument examples and are not authoritative entry points.

11. Future Work

11.1 Adaptive Question Budgets

Select zero, one, two, or more questions according to description richness, existing target facts, and current conflict strength. Detailed descriptions should favor direct comparison, while weak descriptions may reserve more questions for promising candidates.

11.2 Information-Gain Ranking

Estimate the rarity, observability, and expected discrimination of visible candidate attributes, then choose questions programmatically instead of relying only on prompt preferences.

11.3 Stronger Target-Fact Extraction

Convert oracle answers into normalized attribute values, confidence, and negation scope. This would move more cross-candidate comparison from free-form model reasoning into deterministic logic.

11.4 Per-Episode Checkpoints

Atomically write results after each completed episode and load existing IDs at startup. This would enable safe resume and result-shard merging.

11.5 Better Observability

Record false-positive and false-negative categories, rejection reasons, repair counts, answer polarity, and attribute usage frequency for strategy diagnostics.

11.6 Schema-Constrained Output

When a provider supports JSON Schema or structured output, enforce the action contract at the API layer to reduce repair calls and parser branches.

12. Conclusion

The current system combines multimodal model capability with deterministic engineering control. The model interprets images and proposes informative questions; program logic owns evidence provenance, state boundaries, protocol legality, and failure recovery. Structured evidence and bounded context let the questioner accumulate target knowledge across candidates while remaining explainable, testable, and provider-independent.

The architecture provides a clear foundation for adaptive questioning, normalized fact extraction, resumable evaluation, and additional multimodal backends.